Week 8: Database Relationship Analysis using Joins

Scenario:

The e-commerce company stores customer, product, order, and payment information in separate tables. The management team wants to combine these related tables to understand customer purchases, order details, and payment status.

1. Implement INNER JOIN

Concept:

INNER JOIN returns only matching records from both tables.

1. Display customer details along with their orders.

```
SQL> SELECT Customer.CUSTOMER_ID,
  2         Customer.CUSTOMER_NAME,
  3         Orders.ORDER_ID,
  4         Orders.ORDER_DATE,
  5         Orders.TOTAL_AMOUNT,
  6         Orders.ORDER_STATUS
  7  FROM Customer
  8  INNER JOIN Orders
  9  ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID;

CUSTOMER_ID CUSTOMER_NAME     ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
----------- --------------- ---------- --------- ------------ ---------------
----
          2 Arun                  1001 01-OCT-26         2800 Pending
          3 Priya                 1002 02-OCT-26         4500 Shipped
          4 Karthik               1003 03-OCT-26         3400 Delivered
          5 Divya                 1004 04-OCT-26         7500 Shipped
          6 Rahul                 1005 05-OCT-26         1500 Pending
```

2. Display order details with payment information.

```
SQL> SELECT Orders.ORDER_ID,
  2         Orders.ORDER_DATE,
  3         Orders.TOTAL_AMOUNT,
  4         Orders.ORDER_STATUS,
  5         Payment.PAYMENT_ID,
  6         Payment.PAYMENT_MODE,
  7         Payment.PAYMENT_STATUS,
  8         Payment.TRANSACTION_AMOUNT
  9  FROM Orders
 10  INNER JOIN Payment
 11  ON Orders.ORDER_ID = Payment.ORDER_ID;

  ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS         PAYMENT_ID   PAYMENT_MODE    PAYMENT_STATUS  TRANSACTION_AMOUNT
---------- --------- ------------ -------------------- ------------ --------------- --------------- ------------------
      1001 01-OCT-26         2800 Pending              PAY301       UPI             Success                       2800
      1002 02-OCT-26         4500 Shipped              PAY302       Cash            Pending                       4500
      1003 03-OCT-26         3400 Delivered            PAY303       Debit Card      Success                       3400
      1004 04-OCT-26         7500 Shipped              PAY304       Credit Card     Success                       7500
      1005 05-OCT-26         1500 Pending              PAY305       UPI             Success                       1500
```

3. Retrieve products purchased by customers.

```
SQL> SELECT Customer.CUSTOMER_NAME,
  2         Orders.ORDER_ID,
  3         Product.PRODUCT_ID,
  4         Product.PRODUCT_NAME,
  5         Order_Details.QUANTITY,
  6         Order_Details.PRICE
  7  FROM Customer
  8  INNER JOIN Orders
  9  ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID
 10  INNER JOIN Order_Details
 11  ON Orders.ORDER_ID = Order_Details.ORDER_ID
 12  INNER JOIN Product
 13  ON Order_Details.PRODUCT_ID = Product.PRODUCT_ID;

CUSTOMER_NAME     ORDER_ID PRODUCT_ID
--------------- ---------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE
---------- ----------
Arun                 1001 P001
Laptop
         1       2500

Priya                1002 P002
Mobile
         1       4500

CUSTOMER_NAME     ORDER_ID PRODUCT_ID
--------------- ---------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE
```

```
CUSTOMER_NAME     ORDER_ID PRODUCT_ID
--------------- ---------- ----------
PRODUCT_NAME
--------------------------------------------------
  QUANTITY      PRICE
---------- ----------

Karthik              1003 P003
Watch
         1       3400

Divya                1004 P004
Headphones

CUSTOMER_NAME     ORDER_ID PRODUCT_ID
--------------- ---------- ----------
PRODUCT_NAME
--------------------------------------------------
  QUANTITY      PRICE
---------- ----------
         3       2500

Rahul                1005 P005
Bag
         1       1500
```

4. Implement LEFT JOIN

Concept:

LEFT JOIN returns all records from the left table and matching records from the right table. If there is no match, NULL values are returned.

1. Display all customers including customers who have not placed orders.

```
SQL> SELECT Customer.CUSTOMER_ID,
  2         Customer.CUSTOMER_NAME,
  3         Orders.ORDER_ID,
  4         Orders.ORDER_DATE,
  5         Orders.TOTAL_AMOUNT,
  6         Orders.ORDER_STATUS
  7  FROM Customer
  8  LEFT JOIN Orders
  9  ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID;

CUSTOMER_ID CUSTOMER_NAME     ORDER_ID ORDER_DAT TOTAL_AMOUNT
----------- --------------- ---------- --------- ------------
ORDER_STATUS
--------------------
          2 Arun                  1001 01-OCT-26         2800
Pending

          3 Priya                 1002 02-OCT-26         4500
Shipped

          4 Karthik               1003 03-OCT-26         3400
Delivered


CUSTOMER_ID CUSTOMER_NAME     ORDER_ID ORDER_DAT TOTAL_AMOUNT
----------- --------------- ---------- --------- ------------
ORDER_STATUS
--------------------
          5 Divya                 1004 04-OCT-26         7500
Shipped

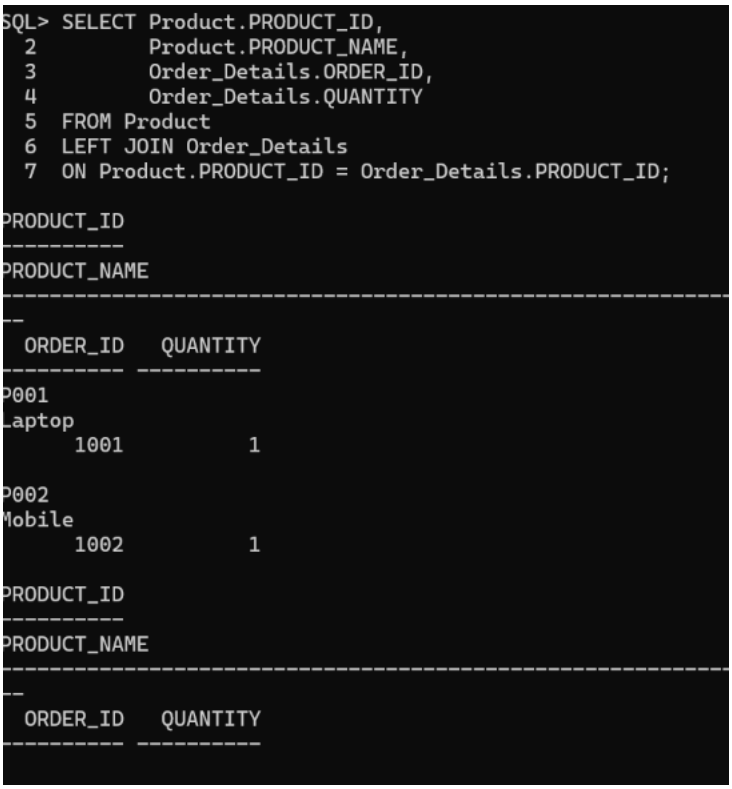
          6 Rahul                 1005 05-OCT-26         1500
Pending
```

2. Find customers without any purchases.

```
SQL> SELECT Customer.CUSTOMER_ID,
  2         Customer.CUSTOMER_NAME
  3  FROM Customer
  4  LEFT JOIN Orders
  5  ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID
  6  WHERE Orders.ORDER_ID IS NULL;

no rows selected
```

3. Display all products including products with no sales.

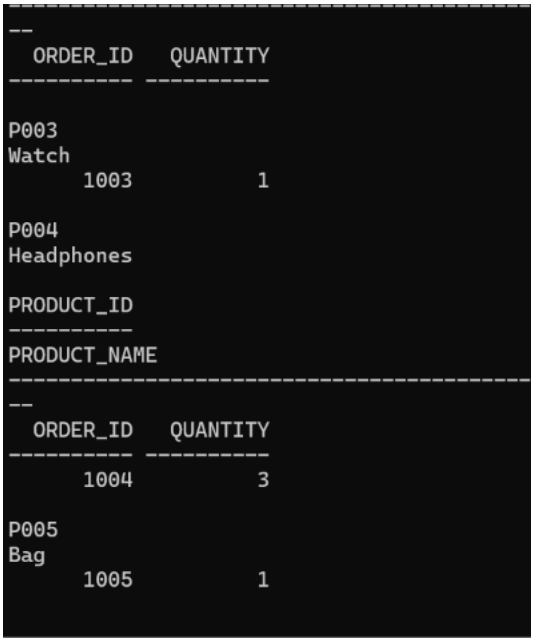


```
SQL> SELECT Product.PRODUCT_ID,
  2         Product.PRODUCT_NAME,
  3         Order_Details.ORDER_ID,
  4         Order_Details.QUANTITY
  5  FROM Product
  6  LEFT JOIN Order_Details
  7  ON Product.PRODUCT_ID = Order_Details.PRODUCT_ID;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------
--
  ORDER_ID   QUANTITY
---------- ----------
P001
Laptop
      1001          1

P002
Mobile
      1002          1

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------
--
  ORDER_ID   QUANTITY
---------- ----------
```



```
--
  ORDER_ID   QUANTITY
---------- ----------

P003
Watch
      1003          1

P004
Headphones

PRODUCT_ID
----------
PRODUCT_NAME
----------------------------------------
--
  ORDER_ID   QUANTITY
---------- ----------
      1004          3

P005
Bag
      1005          1
```

5. Implement RIGHT JOIN

Concept:

RIGHT JOIN returns all records from the right table and matching records from the left table.

1. Display all orders with customer information.

```
SQL> SELECT Orders.ORDER_ID,
  2         Orders.ORDER_DATE,
  3         Orders.TOTAL_AMOUNT,
  4         Orders.ORDER_STATUS,
  5         Customer.CUSTOMER_ID,
  6         Customer.CUSTOMER_NAME
  7  FROM Customer
  8  RIGHT JOIN Orders
  9  ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID;

  ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS         CUSTOMER_ID
---------- --------- ------------ -------------------- -----------
CUSTOMER_NAME
---------------
      1001 01-OCT-26         2800 Pending                        2
Arun

      1002 02-OCT-26         4500 Shipped                        3
Priya

      1003 03-OCT-26         3400 Delivered                      4
Karthik


  ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS         CUSTOMER_ID
---------- --------- ------------ -------------------- -----------
CUSTOMER_NAME
---------------
      1004 04-OCT-26         7500 Shipped                        5
Divya

      1005 05-OCT-26         1500 Pending                        6
Rahul
```

2. Find orders where customer details are missing.

```
SQL> SELECT Orders.ORDER_ID,
  2         Orders.CUSTOMER_ID,
  3         Orders.ORDER_DATE,
  4         Orders.TOTAL_AMOUNT,
  5         Customer.CUSTOMER_NAME
  6  FROM Customer
  7  RIGHT JOIN Orders
  8  ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID
  9  WHERE Customer.CUSTOMER_ID IS NULL;

no rows selected
```

3. Display all payment records with order details.

```
SQL> SELECT Payment.PAYMENT_ID,
  2         Payment.ORDER_ID,
  3         Payment.PAYMENT_DATE,
  4         Payment.PAYMENT_MODE,
  5         Payment.PAYMENT_STATUS,
  6         Payment.TRANSACTION_AMOUNT,
  7         Orders.ORDER_DATE,
  8         Orders.TOTAL_AMOUNT,
  9         Orders.ORDER_STATUS
 10  FROM Orders
 11  RIGHT JOIN Payment
 12  ON Orders.ORDER_ID = Payment.ORDER_ID;

PAYMENT_ID     ORDER_ID PAYMENT_D PAYMENT_MODE    PAYMENT_STATUS
---------- ---------- --------- --------------- ---------------
TRANSACTION_AMOUNT ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
------------------ --------- ------------ --------------------
PAY301             1001 15-OCT-26 UPI             Success
              2800 01-OCT-26         2800 Pending

PAY302             1002 16-OCT-26 Cash            Pending
              4500 02-OCT-26         4500 Shipped

PAY303             1003 17-OCT-26 Debit Card      Success
              3400 03-OCT-26         3400 Delivered


PAYMENT_ID     ORDER_ID PAYMENT_D PAYMENT_MODE    PAYMENT_STATUS
---------- ---------- --------- --------------- ---------------
TRANSACTION_AMOUNT ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
------------------ --------- ------------ --------------------
PAY304             1004 18-OCT-26 Credit Card     Success
              7500 04-OCT-26         7500 Shipped
```

5. Retrieve Complete Order Details

```
SQL> SELECT Customer.CUSTOMER_NAME,
  2         Product.PRODUCT_NAME,
  3         Order_Details.QUANTITY,
  4         Orders.ORDER_DATE,
  5         Orders.TOTAL_AMOUNT,
  6         Payment.PAYMENT_STATUS
  7  FROM Customer
  8  INNER JOIN Orders
  9  ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID
 10  INNER JOIN Order_Details
 11  ON Orders.ORDER_ID = Order_Details.ORDER_ID
 12  INNER JOIN Product
 13  ON Order_Details.PRODUCT_ID = Product.PRODUCT_ID
 14  INNER JOIN Payment
 15  ON Orders.ORDER_ID = Payment.ORDER_ID;

CUSTOMER_NAME
---------------
PRODUCT_NAME
--------------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT PAYMENT_STATUS
---------- --------- ------------ ---------------
Arun
Laptop
         1 01-OCT-26         2800 Success

Priya
Mobile
         1 02-OCT-26         4500 Pending

CUSTOMER_NAME
---------------
PRODUCT_NAME
```

```
CUSTOMER_NAME
---------------
PRODUCT_NAME
--------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT PAYMENT_STATUS
---------- --------- ------------ ---------------

Karthik
Watch
         1 03-OCT-26         3400 Success

Divya
Headphones

CUSTOMER_NAME
---------------
PRODUCT_NAME
--------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT PAYMENT_STATUS
---------- --------- ------------ ---------------
         3 04-OCT-26         7500 Success

Rahul
Bag
         1 05-OCT-26         1500 Success
```

6. Display Customer Purchase History

1. Display all products purchased by a customer.

```
SQL> SELECT Customer.CUSTOMER_NAME,
  2         Product.PRODUCT_NAME,
  3         Order_Details.QUANTITY,
  4         Orders.ORDER_DATE,
  5         Orders.TOTAL_AMOUNT
  6  FROM Customer
  7  INNER JOIN Orders
  8  ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID
  9  INNER JOIN Order_Details
 10  ON Orders.ORDER_ID = Order_Details.ORDER_ID
 11  INNER JOIN Product
 12  ON Order_Details.PRODUCT_ID = Product.PRODUCT_ID
 13  ORDER BY Customer.CUSTOMER_NAME;

CUSTOMER_NAME
---------------
PRODUCT_NAME
------------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT
---------- --------- ------------
Arun
Laptop
         1 01-OCT-26         2800

Divya
Headphones
         3 04-OCT-26         7500

CUSTOMER_NAME
---------------
PRODUCT_NAME
------------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT
```

```
CUSTOMER_NAME
---------------
PRODUCT_NAME
----------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT
---------- --------- ------------

Karthik
Watch
         1 03-OCT-26         3400

Priya
Mobile

CUSTOMER_NAME
---------------
PRODUCT_NAME
----------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT
---------- --------- ------------
         1 02-OCT-26         4500

Rahul
Bag
         1 05-OCT-26         1500
```

2. Display total amount spent by each customer.

```
SQL> SELECT Customer.CUSTOMER_ID,
  2         Customer.CUSTOMER_NAME,
  3         SUM(Orders.TOTAL_AMOUNT) AS TOTAL_SPENT
  4  FROM Customer
  5  INNER JOIN Orders
  6  ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID
  7  GROUP BY Customer.CUSTOMER_ID,
  8           Customer.CUSTOMER_NAME
  9  ORDER BY TOTAL_SPENT DESC;

CUSTOMER_ID CUSTOMER_NAME   TOTAL_SPENT
----------- --------------- -----------
          5 Divya                  7500
          3 Priya                  4500
          4 Karthik                3400
          2 Arun                   2800
          6 Rahul                  1500
```

3. Display the number of orders placed by each customer.

```
SQL> SELECT Customer.CUSTOMER_ID,
  2         Customer.CUSTOMER_NAME,
  3         COUNT(Orders.ORDER_ID) AS TOTAL_ORDERS
  4  FROM Customer
  5  LEFT JOIN Orders
  6  ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID
  7  GROUP BY Customer.CUSTOMER_ID,
  8           Customer.CUSTOMER_NAME
  9  ORDER BY TOTAL_ORDERS DESC;

CUSTOMER_ID CUSTOMER_NAME   TOTAL_ORDERS
----------- --------------- ------------
          3 Priya                      1
          6 Rahul                      1
          5 Divya                      1
          4 Karthik                    1
          2 Arun                       1
```

4. Display the latest purchase details of customers.

```
SQL> SELECT Customer.CUSTOMER_NAME,
  2         Orders.ORDER_ID,
  3         Orders.ORDER_DATE,
  4         Orders.TOTAL_AMOUNT,
  5         Orders.ORDER_STATUS
  6  FROM Customer
  7  INNER JOIN Orders
  8  ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID
  9  WHERE Orders.ORDER_DATE = (
 10      SELECT MAX(O2.ORDER_DATE)
 11      FROM Orders O2
 12      WHERE O2.CUSTOMER_ID = Orders.CUSTOMER_ID
 13  );

CUSTOMER_NAME     ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
--------------- ---------- --------- ------------ --------------------
Arun                  1001 01-OCT-26         2800 Pending
Priya                 1002 02-OCT-26         4500 Shipped
Karthik               1003 03-OCT-26         3400 Delivered
Divya                 1004 04-OCT-26         7500 Shipped
Rahul                 1005 05-OCT-26         1500 Pending
```

7. Generate Multi-Table Business Reports

Report 1 – Customer Order Report

This report displays the customer name, order ID, order date, and order status.

```
SQL> SELECT Customer.CUSTOMER_NAME,
  2         Orders.ORDER_ID,
  3         Orders.ORDER_DATE,
  4         Orders.ORDER_STATUS
  5  FROM Customer
  6  INNER JOIN Orders
  7  ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID
  8  ORDER BY Orders.ORDER_ID;

CUSTOMER_NAME     ORDER_ID ORDER_DAT ORDER_STATUS
--------------- ---------- --------- --------------------
Arun                  1001 01-OCT-26 Pending
Priya                 1002 02-OCT-26 Shipped
Karthik               1003 03-OCT-26 Delivered
Divya                 1004 04-OCT-26 Shipped
Rahul                 1005 05-OCT-26 Pending
```

Report 2 – Sales Report

This report displays the product name, quantity sold, and total revenue.

```
SQL> SELECT Product.PRODUCT_NAME,
  2         SUM(Order_Details.QUANTITY) AS QUANTITY_SOLD,
  3         SUM(Order_Details.QUANTITY * Order_Details.PRICE) AS TOTAL_REVENUE
  4  FROM Product
  5  INNER JOIN Order_Details
  6  ON Product.PRODUCT_ID = Order_Details.PRODUCT_ID
  7  GROUP BY Product.PRODUCT_NAME
  8  ORDER BY TOTAL_REVENUE DESC;

PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
Headphones
            3          7500

Mobile
            1          4500

Watch
            1          3400


PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
Laptop
            1          2500

Bag
            1          1500
```

Report 3 – Payment Analysis Report

This report displays the payment mode, number of transactions, and successful payments.

```
SQL> SELECT PAYMENT_MODE,
  2         COUNT(*) AS NUMBER_OF_TRANSACTIONS,
  3         SUM(CASE
  4             WHEN PAYMENT_STATUS = 'Success' THEN 1
  5             ELSE 0
  6         END) AS SUCCESSFUL_PAYMENTS
  7  FROM Payment
  8  GROUP BY PAYMENT_MODE
  9  ORDER BY PAYMENT_MODE;

PAYMENT_MODE    NUMBER_OF_TRANSACTIONS SUCCESSFUL_PAYMENTS
--------------- ---------------------- ------------------
Cash                                 1                  0
Credit Card                          1                  1
Debit Card                           1                  1
UPI                                  2                  2
```

Report 4 – Customer Purchase Analysis

This report analyses customer purchases to identify top purchasing customers, customers with maximum orders, and customers with the highest spending.

1. Display top purchasing customers.

```
SQL> SELECT Customer.CUSTOMER_ID,
  2         Customer.CUSTOMER_NAME,
  3         SUM(Order_Details.QUANTITY) AS TOTAL_QUANTITY
  4  FROM Customer
  5  INNER JOIN Orders
  6  ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID
  7  INNER JOIN Order_Details
  8  ON Orders.ORDER_ID = Order_Details.ORDER_ID
  9  GROUP BY Customer.CUSTOMER_ID,
 10           Customer.CUSTOMER_NAME
 11  ORDER BY TOTAL_QUANTITY DESC;

CUSTOMER_ID CUSTOMER_NAME   TOTAL_QUANTITY
----------- --------------- --------------
          5 Divya                        3
          6 Rahul                        1
          4 Karthik                      1
          3 Priya                        1
          2 Arun                         1
```

2. Display customers with the maximum number of orders.

```
SQL> SELECT Customer.CUSTOMER_ID,
  2         Customer.CUSTOMER_NAME,
  3         COUNT(Orders.ORDER_ID) AS TOTAL_ORDERS
  4  FROM Customer
  5  INNER JOIN Orders
  6  ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID
  7  GROUP BY Customer.CUSTOMER_ID,
  8           Customer.CUSTOMER_NAME
  9  HAVING COUNT(Orders.ORDER_ID) = (
 10      SELECT MAX(ORDER_COUNT)
 11      FROM (
 12          SELECT COUNT(*) AS ORDER_COUNT
 13          FROM Orders
 14          GROUP BY CUSTOMER_ID
 15      )
 16  );

CUSTOMER_ID CUSTOMER_NAME   TOTAL_ORDERS
----------- --------------- ------------
          3 Priya                      1
          6 Rahul                      1
          2 Arun                       1
          4 Karthik                    1
          5 Divya                      1
```

3. Display customers with the highest spending.

```
SQL> SELECT Customer.CUSTOMER_ID,
  2         Customer.CUSTOMER_NAME,
  3         SUM(Orders.TOTAL_AMOUNT) AS TOTAL_SPENT
  4  FROM Customer
  5  INNER JOIN Orders
  6  ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID
  7  GROUP BY Customer.CUSTOMER_ID,
  8           Customer.CUSTOMER_NAME
  9  HAVING SUM(Orders.TOTAL_AMOUNT) = (
 10      SELECT MAX(TOTAL_SPENT)
 11      FROM (
 12          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPENT
 13          FROM Orders
 14          GROUP BY CUSTOMER_ID
 15      )
 16  );

CUSTOMER_ID CUSTOMER_NAME   TOTAL_SPENT
----------- --------------- -----------
          5 Divya                  7500
```

RESULT

The SQL queries were executed using SQL*Plus on the E-Commerce Database. INNER JOIN, LEFT JOIN, and RIGHT JOIN operations were performed to retrieve related information from multiple tables. Customer purchase history and multi-table business reports were generated to analyse orders, sales, and payment transactions.

CONCLUSION

Thus, the required database relationship analysis was performed successfully using SQL*Plus. The practical demonstrated the use of INNER JOIN, LEFT JOIN, RIGHT JOIN,

aggregate functions, subqueries, and multi-table joins to generate meaningful business reports.